

Arquitetura de Software como Representação em Camadas

Análise de Arquitetura de Software com S202

Johannes Weigend

Versão final, 30 de maio de 2026

Resumo

S202 calcula, a partir de bytecode, uma representação hierárquica em camadas de sistemas Java. A ferramenta deriva uma hipótese arquitetural das dependências observadas entre classes, distingue dois tipos de ciclos – ciclos de classes e ciclos de pacotes –, torna ambos visíveis como achados ativados separadamente e verifica a representação final de forma redundante contra regras de consistência. Este documento descreve erros típicos no cálculo desses layouts, explica a solução conceitual e mostra como a representação pode ser usada como ferramenta de trabalho para planejamento de refatoração.

1 Introdução

Pelo menos desde os modelos em camadas para sistemas operacionais no fim da década de 1960, a arquitetura de software vem sendo representada como uma estrutura de dependências em camadas. Esse princípio continua atual: em arquiteturas cloud native ele ajuda a ordenar dependências técnicas entre serviços, plataformas e infraestrutura; no domain-driven design, ajuda a tornar visíveis os limites de domínio e as dependências entre bounded contexts. A ideia básica é simples: um elemento que usa outro elemento fica acima do elemento usado. As dependências, portanto, correm de cima para baixo. Arestas na direção oposta chamam a atenção porque muitas vezes indicam ciclos, acoplamento indesejado ou uma ordem em camadas quebrada.

Grafos alinhados verticalmente desse tipo ajudam a entender sistemas e a reconhecer anomalias em suas dependências. Um exemplo simples é fornecido pela ferramenta Java `jdeps` e pela ferramenta Graphviz `dot`: `jdeps` extrai as dependências entre classes de um JAR, e `dot` gera a partir delas um grafo dirigido.

A Figura 1 mostra um exemplo acíclico do Example JAR da base de código do S202. Ele foi criado com `jdeps` e `dot`. O JAR contém nove classes em três pacotes: `example`, `example1` e `example2`.

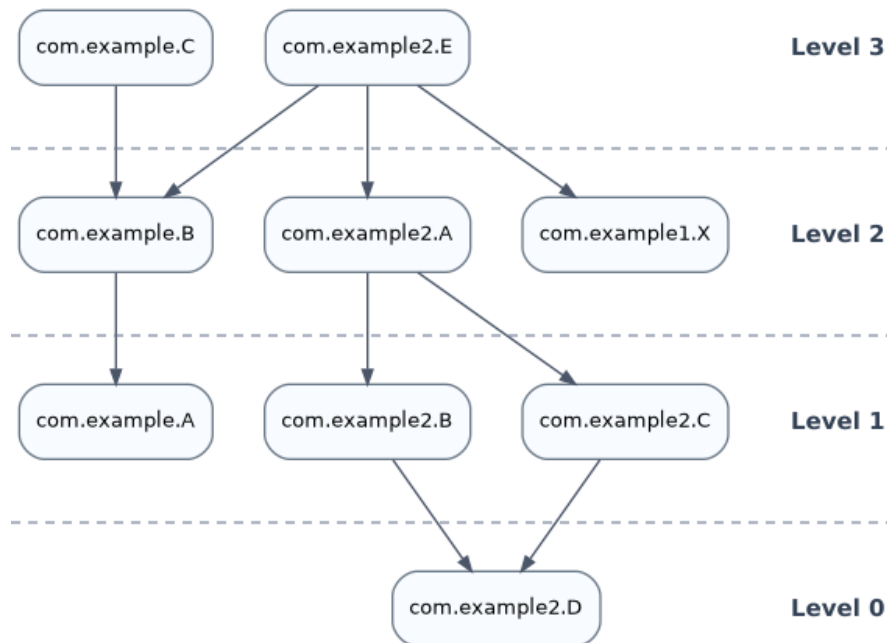


Figura 1: Exemplo acíclico de `test-example-1.0.0.jar`.

As setas entre as caixas são dependências entre classes por chamada ou uso. No exemplo, `com.example.B` usa a classe `com.example.A`, de forma abreviada $B \rightarrow A$. As linhas horizontais foram acrescentadas manualmente e visualizam os níveis da camada. O Level 0 forma a base; níveis mais altos representam elementos que dependem de elementos abaixo deles.

Esse tipo de representação é útil e amplamente usado. Como visualização de aplicações reais, porém, ele tem três problemas práticos:

1. Muitas setas tornam o grafo confuso já em aplicações de porte médio.
2. A estrutura de pacotes fica distribuída por vários níveis e não forma um agrupamento visível.
3. Em dependências cíclicas, o ciclo precisa ser quebrado para o cálculo dos níveis, para que surja alguma ordem vertical. O grafo original de dependências permanece inalterado; muda apenas o DAG derivado (*Directed Acyclic Graph*, um grafo dirigido sem ciclos) sobre o qual os níveis são calculados. A questão decisiva, portanto, é qual aresta fica fora desse cálculo, pois exatamente essa aresta aparece depois como quebra da ordem calculada.

2 S202 como Representação Hierárquica em Camadas

Com a ferramenta open source S202, buscamos tratar esses problemas de forma direta. S202 não substitui simplesmente o grafo plano de classes por uma representação melhorada. Ele o substitui por uma hipótese arquitetural hierárquica: pacotes permanecem visíveis como contêineres e, dentro de cada contêiner, pacotes ou classes são ordenados de acordo com suas dependências.

As setas podem ser exibidas opcionalmente. Normalmente, porém, a ordem vertical já deve deixar compreensível quais partes da aplicação ficam mais acima e quais servem de base. Dependências

indesejadas, assim, não aparecem apenas como uma linha isolada em uma grande rede, mas como uma quebra em uma ordem esperada de camadas.

Ao contrário do grafo plano de classes, em S202 os níveis são estruturados hierarquicamente pela estrutura de pacotes. Cada pacote ordena seus filhos diretos, isto é, subpacotes ou classes, segundo suas dependências locais. A estrutura de pacotes permanece visível, em vez de ser espalhada pelo grafo de classes.

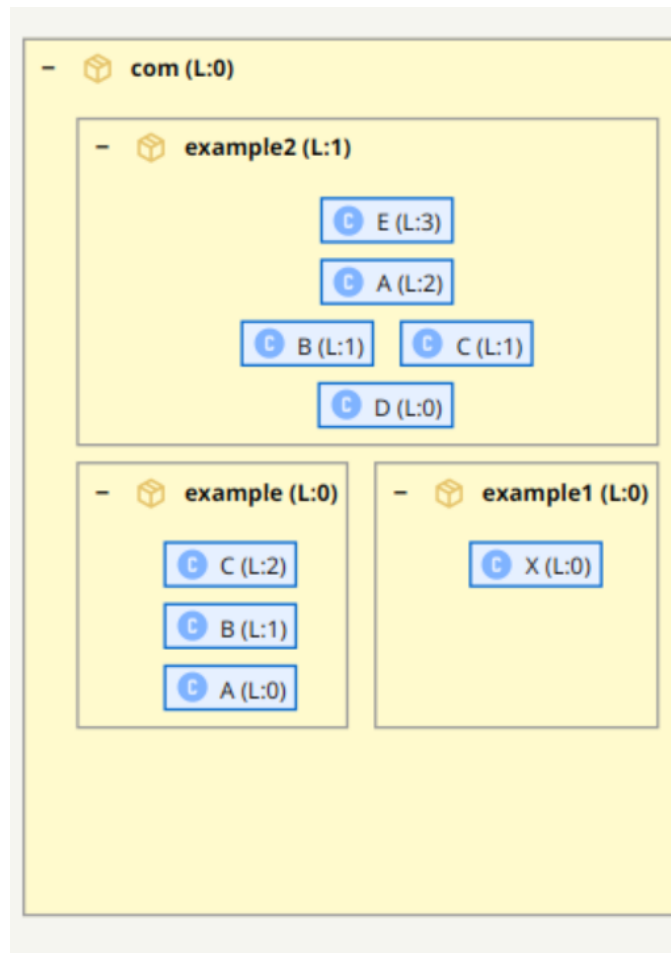


Figura 2: Representação S202 do exemplo acíclico: pacotes permanecem visíveis como contêineres, classes são ordenadas dentro de seu contêiner e setas podem ser exibidas quando necessário.

O exemplo mostra a estrutura de pacotes abaixo do pacote externo `com`: `example`, `example1` e `example2` são subárvores desse contêiner. Na subárvore `example2` existe uma estrutura interna de classes não trivial; ao mesmo tempo, a estrutura mais ampla mostra que `example2` depende de `example` e `example1`. No nível de classes vale, para `example2`, $E \rightarrow A \rightarrow (B \mid C) \rightarrow D$. A representação abstrai deliberadamente das arestas individuais. Se `A` tem uma dependência para `B`, para `C` ou para ambos é menos importante para a visão arquitetural ampla do que a estrutura em camadas do pacote. Exatamente essa abstração torna redes extensas de relações ainda legíveis.

A ideia não é nova. O produto comercial Structure101 já tinha uma representação muito semelhante. No entanto, não há uma implementação aberta nem uma documentação técnica que explique como calcular esse layout de forma confiável. S202 fecha essa lacuna como ferramenta open source sob a licença Apache.

Este documento descreve as armadilhas típicas na realização de um layout desse tipo e mostra uma solução com a qual muitos desses problemas podem ser eliminados de forma conceitualmente

limpa.

3 Erros Típicos no Cálculo de Camadas

As Figuras 1 e 2 mostram deliberadamente um exemplo simples e acíclico. Nesse caso, a camada é inequívoca: E fica acima de A, A fica acima de B e C, e ambos ficam acima de D. Não há realimentação, portanto nenhuma aresta precisa ser escolhida para ser ignorada no cálculo dos níveis.

Em aplicações reais esse caso é mais exceção do que regra. Assim que classes ou pacotes dependem ciclicamente uns dos outros, uma passagem normal pelo grafo não basta mais. É exatamente aí que começam os erros interessantes: a representação muitas vezes ainda parece plausível, mas os níveis calculados deixam de ser estáveis ou deixam de ser explicáveis em termos arquiteturais.

Nas seções seguintes, o termo SCC tem papel central. Uma SCC (*Strongly Connected Component*, componente fortemente conexa) é um conjunto de nós no qual cada nó pode alcançar todos os outros por caminhos dirigidos de dependência. Em um ciclo desse tipo, nem todas as dependências podem correr para baixo ao mesmo tempo.

Um layout em camadas correto precisa, portanto, satisfazer vários requisitos ao mesmo tempo:

1. Se $A \rightarrow B$, então A deve ficar acima de B .
2. Pacotes precisam permanecer visíveis como contêineres. Um pacote não deve ser deslocado por acaso apenas porque uma classe contida nele tem um nível de classe alto.
3. Classes ou pacotes que formam uma SCC precisam ser tratados explicitamente, porque em um ciclo nem todas as arestas podem correr para baixo ao mesmo tempo.

Esses requisitos são individualmente bem resolvíveis. Juntos, porém, eles criam um sistema circular de dependências entre os próprios níveis da computação: níveis de classes influenciam posições visíveis, contêineres de pacotes estruturam essas posições, e SCCs forçam correções em uma ordem que já teria sido calculada sem o tratamento de ciclos. É daí que surgem os modos típicos de falha.

3.1 Correção Retroativa

Correção retroativa significa aqui: uma estrutura reconhecida mais tarde obriga uma alteração posterior em níveis já calculados.

Um algoritmo ingênuo percorre as classes e atribui níveis diretamente. Ele vê primeiro a classe A, define seu nível e com isso também eleva o pacote de A. Mais tarde encontra a classe B e reconhece: A e B pertencem à mesma SCC e precisam ser tratadas em conjunto. Agora A teria de ser corrigida, e com ela seu pacote e possivelmente outros pacotes que dependem dele.

O problema não é a correção em si. O problema é que partes do layout, nesse momento, já foram tratadas como concluídas. Um algoritmo que mistura detecção de ciclos e atribuição de níveis precisa alterar retroativamente lugares que ele na verdade já deixou para trás.

3.2 Resultados Dependentes da Ordem

Classes vêm de JARs. A ordem em que uma ferramenta vê classes não segue a arquitetura, mas a ordem de empacotamento do JAR, a ordem dos módulos, a ferramenta de build ou detalhes

do sistema de arquivos. Se um algoritmo trata ciclos incidentalmente durante essa travessia, o resultado depende dessa ordem acidental.

Isso é perigoso porque o erro não precisa ser visível imediatamente. O grafo ainda pode parecer aproximadamente correto. Mesmo assim, a mesma base de código pode receber níveis diferentes dependendo da configuração de build.

3.3 Mistura de Níveis de Classe e de Pacote

Um erro natural é derivar o nível de um pacote a partir do maior nível de suas classes contidas. Isso parece plausível, mas mistura duas perguntas diferentes: onde uma classe fica no grafo de classes e onde um pacote fica na hipótese arquitetural.

Um pacote não está arquiteturalmente alto apenas porque por acaso contém uma classe posicionada alto. A regra central é:

Níveis de pacotes surgem de dependências entre pacotes, não da altura máxima de classes individuais.

Isso parece trivial, mas não é. Justamente porque pacotes são contêineres de classes, parece natural derivar sua posição das classes contidas. Mas é exatamente assim que se misturam dois níveis que precisam permanecer separados. Somente quando níveis de pacotes surgem de dependências entre pacotes, os pacotes permanecem unidades arquiteturais estáveis. Caso contrário, uma única classe fora do padrão basta para elevar o pacote ao nível errado e tornar inutilizável o contêiner que deveria oferecer orientação.

3.4 O Grande Lago

A resposta formalmente limpa para ciclos é: todos os elementos de uma SCC recebem o mesmo nível. Para ciclos pequenos isso é aceitável. Em sistemas reais, porém, SCCs podem ficar muito grandes. Então centenas ou milhares de classes acabam no mesmo nível. A camada desaparece; o que resta é um bloco plano.

O ciclo, assim, foi reconhecido corretamente, mas deixou de ser visualizado de forma útil. Uma ferramenta precisa, portanto, conseguir quebrar SCCs grandes. O ponto decisivo é que as arestas cortadas não podem desaparecer. Elas precisam permanecer visíveis, porque explicam por que a ordem calculada vale apenas como hipótese arquitetural.

4 Dois Tipos de Ciclos

S202 distingue dois tipos fundamentalmente diferentes de ciclos e torna ambos visíveis como overlays ativados separadamente:

Ciclos de classes (vermelho) Componentes fortemente conexas no nível de classes. Duas ou mais classes que dependem uma da outra direta ou transitivamente formam tal ciclo. Ciclos de classes revelam acoplamento no nível de implementação que impede uma ordem linear estrita de classes individuais. Eles são detectados pelo algoritmo de Tarjan sobre o grafo de dependências de classes.

Ciclos de pacotes (laranja) Tangles no nível de pacotes, detectados sobre o grafo agregado direto de pacotes (sem a propagação de ancestrais descrita adiante). Um ciclo de pacotes significa que dois ou mais pacotes dependem uns dos outros como grupo, independentemente de quais classes concretas são responsáveis. Ciclos de pacotes revelam acoplamento no nível arquitetural: a própria estrutura de pacotes tem dependências circulares.

A distinção importa porque os dois tipos respondem a perguntas diferentes:

- Um *ciclo de classes* mostra quais classes concretas precisam ser desembaralhadas. Dissolvê-lo frequentemente exige mover um método, extrair uma interface ou inverter uma dependência entre duas classes específicas.
- Um *ciclo de pacotes* mostra onde os próprios limites de pacotes estão quebrados. Resolvê-lo pode exigir uma reestruturação maior – como mesclar dois pacotes, introduzir um novo, ou redefinir inteiramente a divisão de módulos.

Um ciclo de pacotes pode existir mesmo quando nenhum ciclo de classes individual é visível – por exemplo, quando dois pacotes se chamam mutuamente por meio de classes diferentes que são individualmente acíclicas. Por outro lado, um ciclo de classes inteiramente contido em um único pacote não implica necessariamente um ciclo de pacotes.

Como os dois overlays respondem a perguntas diferentes, S202 os mantém como chaves independentes. Ativar ambos ao mesmo tempo é possível e útil ao buscar a causa raiz de um problema arquitetural, mas cada overlay também é significativo por si só: o overlay de ciclos de pacotes revela a forma estrutural do problema, enquanto o overlay de ciclos de classes revela as arestas concretas que o causam.

5 O Grafo de Pacotes como Hipótese Arquitetural

A solução escolhida pelo S202 consiste em separar de forma consequente a análise, a formação da ordem e a representação. O grafo de classes permanece o modelo bruto das dependências reais. Ele não é alterado apenas porque uma representação em camadas precisa de um DAG. Em vez disso, S202 cria a partir desse grafo visões adicionais: SCCs para a detecção de ciclos, um grafo de pacotes ponderado para a hipótese arquitetural e uma estrutura hierárquica de UI para a representação.

Com hipótese arquitetural não se quer dizer uma arquitetura-alvo afirmada. Quer-se dizer uma arquitetura real inferida das dependências observadas: a ordem que melhor corresponde ao código existente e a suas dependências sem esconder onde essa ordem é quebrada por ciclos ou back edges.

A ordem de pacotes forma o elo central. Ela não é derivada dos maiores níveis de classes, mas das dependências agregadas entre pacotes. Essa separação é o passo decisivo: classes são ordenadas dentro de seus pacotes; pacotes, por sua vez, são ordenados por suas próprias relações entre pacotes. Assim, um pacote pode permanecer estável como contêiner, mesmo que classes individuais dentro dele fiquem altas ou baixas. Ao mesmo tempo, a ordem de pacotes fornece um critério para escolher back edges não arbitrariamente, mas como violações concretas da ordem esperada.

Com isso, os modos de falha descritos acima não desaparecem por meio de um truque algorítmico isolado, mas por uma responsabilidade clara das etapas de cálculo: primeiro as dependências são analisadas, depois uma ordem plausível é determinada, em seguida a estrutura visível é construída e, por fim, verifica-se se a representação cumpre sua própria afirmação.

6 De Dependências de Classes à Ordem de Pacotes

O grafo plano de classes não é, em S202, o modelo visível de layout. Ele é um artefato de análise: serve para registrar dependências concretas, reconhecer SCCs, agregar dependências de classes em relações entre pacotes e derivar delas os níveis posteriores. A representação S202 visível surge depois da estrutura de pacotes e da ordem local dentro dos contêineres de pacotes.

A ordem de pacotes surge de um grafo dirigido ponderado que abrange *todos* os pacotes. Classes não são nós de comparação nessa etapa; elas fornecem a informação para a agregação. Para cada dependência de classe, S202 conta as chamadas de método da classe de origem para a classe de destino e atribui essa contagem à aresta de pacote correspondente: se uma classe na subárvore do pacote P chama uma classe na subárvore do pacote Q via n chamadas de método, a aresta $P \rightarrow Q$ recebe peso n . Se uma classe tem apenas dependências sem contagem de chamadas para um pacote de destino, ela contribui com um peso de fallback de 1 para essa aresta de pacote (por classe e pacote de destino, não por dependência).

Para capturar a contribuição de pacotes aninhados, cada peso é propagado pela cadeia de ancestrais do pacote de origem: uma chamada de `com.a.X` para `com.b.Y` contribui com peso para `com.a→com.b` e também para `com→com.b`. A ordem transitiva entre pacotes surge apenas no cálculo de níveis subsequente – nenhuma aresta transitiva adicional é inserida no grafo.

A propagação de ancestrais pode, porém, criar ciclos que não existem no grafo direto de pacotes. Ciclos de pacotes (tangles) são, portanto, detectados sobre o grafo direto, não propagado; a quebra de ciclos e a atribuição de níveis, em contrapartida, operam sobre o grafo propagado.

Um exemplo numérico concreto: as dependências de classe `com.a.A→net.b.B` e `com.a.B→net.b.B` geram as seguintes arestas no grafo de pacotes:

```
com.a  -(2)-> net.b
com    -(2)-> net.b
```

Apenas o lado da origem é propagado pela cadeia de ancestrais; o lado do destino permanece no pacote pai direto da classe alvo. `net.b` não é propagado adiante para `net`. Propagar também o lado do destino geraria um produto cartesiano de ancestrais de origem e destino para cada dependência de classe: o número de arestas cresce quadraticamente com a profundidade dos pacotes, sem ganho de informação para o cálculo da ordem.

Antes do cálculo dos níveis, o grafo de pacotes precisa ser acíclico. Se pacotes formam uma SCC, S202 usa uma medida de rank para avaliar se um pacote tende mais a usar outros pacotes ou a ser usado por eles:

$$rank(P) = \frac{out(P) - in(P)}{\max(1, out(P) + in(P))}$$

$out(P)$ é a soma dos pesos das arestas de P para outros membros da mesma SCC, e $in(P)$ é a soma dos pesos das arestas desses membros para P . Um rank positivo significa: o pacote usa mais do que ele mesmo é usado; ele tende a pertencer mais acima. Um rank negativo significa: o pacote é mais usado e tende a pertencer mais abaixo.

Em uma SCC de pacotes, todas as arestas $P \rightarrow Q$ para as quais $rank(P) < rank(Q)$ vale são tratadas como back edges – ou seja, todas as arestas que correm contra o fluxo de dependências. Nenhuma tolerância é utilizada: qualquer diferença mensurável de rank é justificativa suficiente para um corte.

Se todos os ranks dentro de uma SCC são iguais, a topologia não fornece informação de direção. Nesse caso, todas as arestas internas são removidas e os níveis dos nós afetados surgem

exclusivamente de dependências fora da antiga SCC. Se tampouco há essas dependências, os nós permanecem no mesmo nível – como pares genuínos sem ordem justificável.

Os dois tipos de corte são aplicados iterativamente: após cada passagem, as SCCs são recalculadas e, para SCCs residuais, os ranks são recalculados sobre seus membros. O procedimento termina quando não resta nenhuma SCC com pelo menos dois membros.

A detecção de simetria baseia-se na medida de rank, não na comparação de pesos. Uma SCC em que todas as arestas têm o mesmo peso não é necessariamente simétrica: graus de entrada e saída diferentes produzem ranks diferentes apesar de pesos iguais e, assim, indicam uma direção clara de corte.

Depois desse tratamento de ciclos, S202 calcula os níveis sobre o DAG resultante. O Level 0 contém os pacotes que não usam nenhum outro pacote. Todo outro pacote fica um nível acima do pacote mais alto que ele usa diretamente. Assim surge a estratificação transitiva: de $B \rightarrow A$ e $C \rightarrow B$ segue que A fica no Level 0, B no Level 1 e C no Level 2, mesmo sem existir uma aresta direta $C \rightarrow A$.

Essa regra – conhecida como nivelamento de *caminho mais longo* ou *ASAP* – ancora cada pacote o mais baixo possível, imediatamente acima de sua dependência mais profunda. Ela coincide com a invariante que o Structure101 documenta para seus Levelized Structure Maps: cada elemento depende de pelo menos um elemento no nível imediatamente abaixo. A alternativa de puxar cada pacote rente *abaixo de seu consumidor* (ALAP) violaria essa invariante, pois poderia surgir uma lacuna entre um pacote e sua dependência mais próxima.

Uma consequência é aceita deliberadamente: pacotes com o mesmo papel, mas com cadeias de dependência de profundidade diferente, acabam em níveis diferentes. Se, por exemplo, vários adaptadores ficam acima da mesma camada de aplicação, mas um deles fica adicionalmente acima de um pacote auxiliar compartilhado, esse é colocado um nível acima dos demais – o nivelamento mede profundidade de dependência, não papel arquitetural.

Os níveis produzidos aqui são o *architectureLevel* de cada pacote – a hipótese arquitetural que guia o tratamento de SCCs de classes na etapa seguinte. Em uma etapa separada, executada após os níveis de classes serem atribuídos, dentro de cada contêiner pai é construído de forma independente um grafo ponderado somente entre irmãos dos filhos diretos. As back edges de classes já resolvidas na etapa anterior são excluídas, de modo que o grafo de irmãos é acíclico por construção. Desse grafo é derivado um *localLevel* para cada elemento. O localLevel controla o posicionamento visual dentro do contêiner pai e não influencia a hipótese arquitetural nem a detecção de violações.

7 Do Ciclo de Classes à Ordem de Pacotes

A Figura 3 mostra um exemplo cíclico do Example JAR como grafo clássico de classes. Os dois subgrafos para Notification e Payment contêm realimentações.

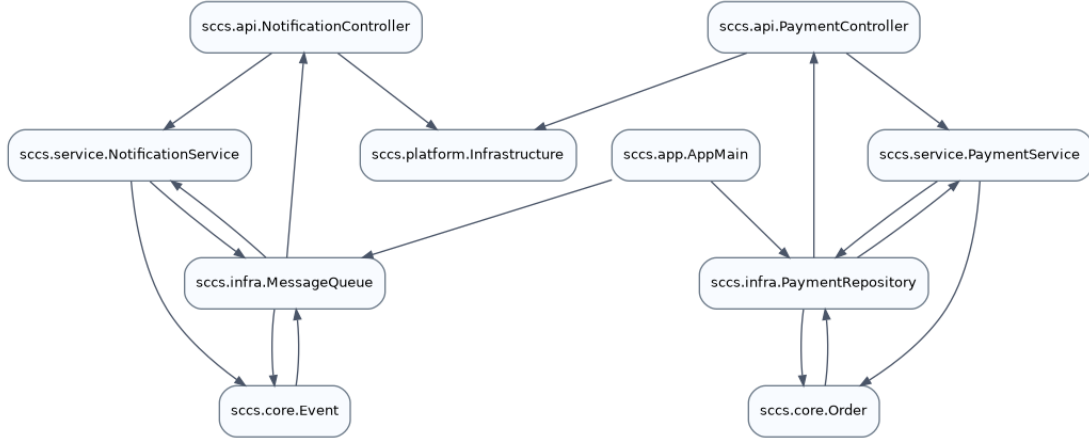


Figura 3: Exemplo cíclico de `test-example-1.0.0.jar`. Em cada ciclo, pelo menos uma aresta precisa ser cortada para o cálculo dos níveis.

As regiões cíclicas marcadas são duas SCCs não triviais: uma SCC de Notification com `NotificationController`, `NotificationService`, `MessageQueue` e `Event`, e uma SCC de Payment com `PaymentController`, `PaymentService`, `PaymentRepository` e `Order`. O procedimento estabelecido para encontrar esses componentes é o algoritmo de Tarjan.

No caso cíclico, o grafo de classes coloca a pergunta técnica: qual aresta precisa ser ignorada para que a SCC volte a ser um DAG calculável para a formação dos níveis? Mas o grafo de classes sozinho não fornece um critério arquitetural para isso. A ordem de pacotes fornece esse critério por meio de dois casos.

Caso A abrange SCCs cujos membros estão distribuídos por diferentes níveis de pacotes. Uma aresta $A \rightarrow B$ dentro de tal SCC é cortada quando $pkgLevel(A) < pkgLevel(B)$ vale – ela corre de um pacote posicionado mais baixo para um mais alto, contra a direção arquitetural. O corte é aplicado iterativamente até que nenhuma aresta dentro de uma SCC corra contra a hipótese de pacotes; SCCs que restam depois disso são tratadas pelo Caso B.

Caso B abrange SCCs confinadas inteiramente a um único nível de pacote – nenhum contexto de pacote fornece uma direção. Todas as arestas internas de tal SCC são removidas. Os níveis de classes dos antigos membros do ciclo surgem então de suas dependências fora da SCC; nós sem tais dependências externas ficam no mesmo nível.

Todas as arestas removidas em ambos os casos são registradas como achados de *classBackEdge* e permanecem visíveis como arestas tracejadas de violação.

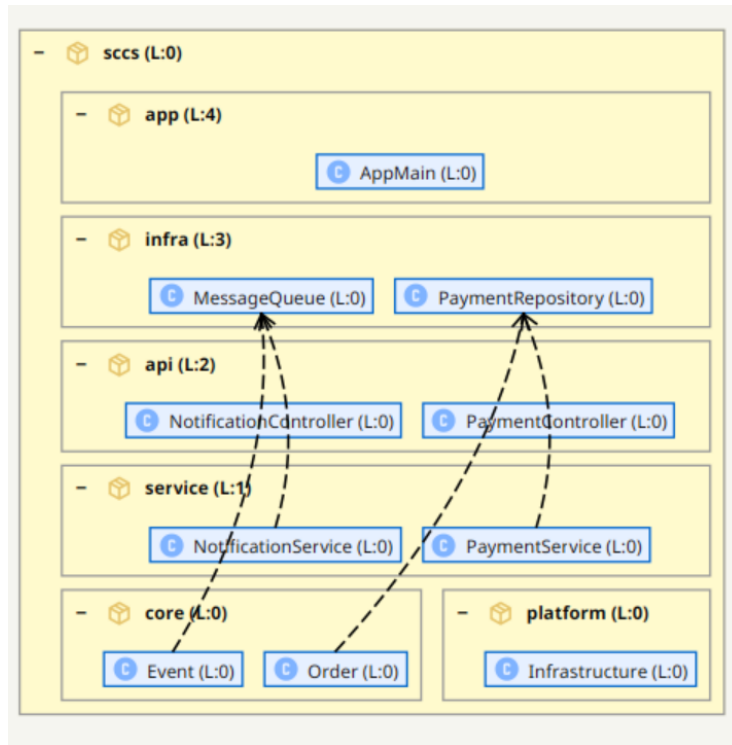


Figura 4: Exemplo cíclico na representação S22. As arestas tracejadas são violações: elas correm contra a ordem derivada das dependências entre pacotes e permanecem como achados visíveis da hipótese arquitetural. Todas as quatro arestas tracejadas pertencem às duas SCCs e são classificadas pelo S22 como back edges para o cálculo dos níveis.

Os termos back edge e violação descrevem papéis diferentes:

Back edge Uma aresta escolhida dentro de uma SCC que é tratada separadamente no cálculo dos níveis para que o grafo de cálculo se torne acíclico. Ela pertence ao tratamento de ciclos.

Violação Uma dependência concreta que, depois da ordem calculada, corre para cima. O que conta aqui é a ordem *aninhada*, não o nível global plano: as cadeias de *localLevel* dos dois extremos são comparadas a partir do contêiner comum mais externo para baixo, e a direção é decidida pelo primeiro contêiner em que diferem – ou seja, o *localLevel* do nível-alvo apropriado (chamado *visual rank* no código). Uma aresta pode correr para baixo globalmente (pelo *architectureLevel*) e ainda assim apontar para cima entre dois pacotes irmãos – e nesse caso é uma violação. Ela pode vir de uma SCC, mas não precisa. Violações são portanto um superconjunto dos back edges: um back edge pode tornar-se visível como violação, mas nem toda violação é um back edge. Conceitualmente, a violação continua sendo o desvio visível da ordem, enquanto um back edge descreve o papel da aresta no tratamento de ciclos.

S22, portanto, não esconde o ciclo. Arestas classificadas como back edges permanecem visíveis como violações porque explicam por que a ordem de pacotes não corresponde completamente às dependências concretas entre classes. Assim, o desvio se torna rastreável em vez de arbitrário.

8 Verificação Redundante da Consistência

As etapas descritas até aqui já mostram por que a implementação é propensa a erros. Não basta proteger algoritmos isolados com unit tests. Muitos erros surgem apenas da interação dos dados: dependências concretas entre classes, estrutura de pacotes, SCCs, pesos agregados de pacotes e posições locais de representação influenciam-se mutuamente.

Por isso S202 não verifica apenas passos individuais de cálculo. Depois que toda a pipeline rodou, o resultado final é verificado de forma redundante contra regras de consistência. Essas regras não recalculam o layout e não corrigem nada. Elas apenas leem o resultado e perguntam: a representação gerada é consistente em si mesma?

A verificação considera três níveis:

1. **Grafo de classes:** Para dependências normais entre classes deve valer: se $A \rightarrow B$, então A está em um nível mais alto que B . Exceções precisam ser classificadas explicitamente, por exemplo como back edge ou como aresta dentro de uma SCC de classes. Ciclos detectados no nível de classes correspondem ao overlay vermelho na interface.
2. **Grafo de pacotes:** A ordem de pacotes calculada precisa ser consistente com as dependências ponderadas entre pacotes. Para cada dependência de pacote $P \rightarrow Q$ que não está classificada como back edge de pacote, P deve ficar estritamente acima de Q na ordem calculada. Arestas cortadas ao quebrar uma SCC de pacotes precisam ser classificadas como back edges de pacotes e estão isentas dessa verificação. Ciclos detectados no nível de pacotes correspondem ao overlay laranja na interface.
3. **Representação visível:** A disposição visível precisa corresponder ao modelo calculado. Contêineres de pacotes, posições locais de classes e níveis aninhados não podem inverter silenciosamente a hipótese arquitetural calculada.

Essas regras de consistência não são uma otimização adicional nem um segundo algoritmo de layout. Elas são uma verificação independente de plausibilidade após o fim do cálculo. Exatamente por isso encontram erros que faltam facilmente em unit tests normais: um nível de pacote foi calculado corretamente, mas exibido localmente de forma errada; um back edge foi cortado, mas não marcado como tal; uma aresta normal corre para cima sem ser parte de uma SCC.

O ponto decisivo é a redundância. A pipeline gera a representação. As regras verificam em seguida se essa representação cumpre sua própria afirmação. Com isso, uma simples figura se torna um modelo verificável.

9 Da Visualização à Ação

Uma hipótese arquitetural calculada é especialmente útil quando não apenas descreve um estado, mas torna mudanças planejáveis. Para isso, S202 contém uma análise what-if: classes e pacotes podem ser movidos por drag and drop na representação. O modelo não é refatorado imediatamente; em vez disso, a ferramenta calcula quais dependências passariam a correr para cima na nova disposição.

As violações se atualizam após cada deslocamento. Assim fica visível se uma reordenação planejada torna a arquitetura mais consistente ou cria novos problemas. Isso é importante sobretudo em situações nas quais a ferramenta não encontra mais uma ordem inequívoca: SCCs grandes, dependências simétricas entre pacotes ou acoplamentos historicamente crescidos nem sempre podem ser resolvidos automaticamente de maneira sensata.

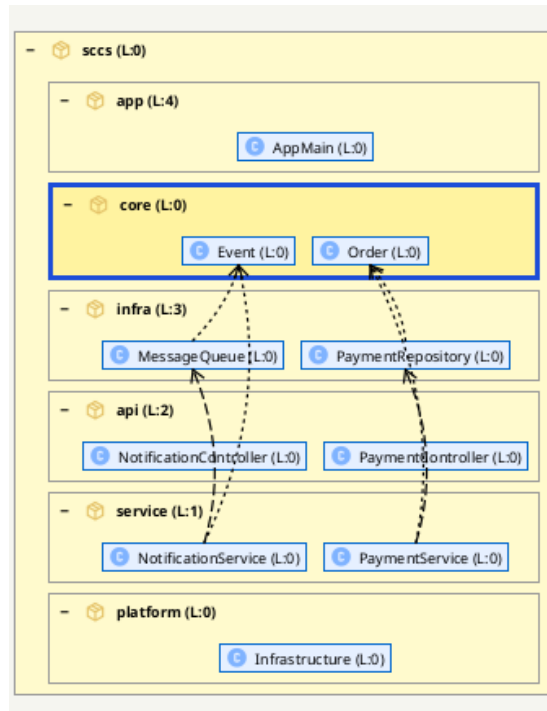


Figura 5: Movimento what-if: o pacote `core` é movido do Level 0 para entre o Level 3 e o Level 4. Isso cria seis violações, porque dependências existentes agora correm contra a disposição escolhida.

A Figura 5 mostra exatamente essa forma de trabalho: a disposição manual já não corresponde à ordem calculada a partir das dependências. Ela pode, mesmo assim, expressar uma arquitetura-alvo desejada, por exemplo se `core` deve ficar conceitualmente acima da infraestrutura. S202, portanto, não trata essa decisão como erro da representação, mas torna visível seu custo: nesse caso, seis relações concretas teriam de ser refatoradas ou desacopladas para que as dependências voltassem a combinar com a camada escolhida.

Complementarmente, existe uma lógica manual de CUT. Back edges podem ser marcados de forma direcionada como cortes planejados, e a ferramenta pode em seguida verificar se esses cortes realmente dissolvem a SCC afetada. O usuário não vê apenas qual aresta seria uma quebra plausível da ordem, mas também pode verificar o efeito desse corte sobre o ciclo.

Essa lógica de CUT trabalha em granularidade mais fina que a aresta visível entre classes ou pacotes. O corte ocorre no nível de chamada de método: não necessariamente desaparece toda a relação entre duas classes, mas a chamada concreta de método que causa a dependência cíclica. Assim o recurso se ajusta melhor ao trabalho prático de refatoração, porque um ciclo muitas vezes surge de poucas chamadas concretas, não da relação funcional completa entre duas classes.

10 Implementação em Nível Conceitual

A implementação separa conscientemente o cálculo em várias fases. O ponto decisivo é que nenhuma fase fixa uma posição visível antes que estejam disponíveis as informações que justificam essa posição.

1. **Construir o grafo de classes:** A partir do bytecode é criado um grafo dirigido de classes. Esse grafo permanece o modelo bruto de domínio da análise. Apenas arestas que carregam

pelo menos uma relação do tipo **extends**, **implements**, **imports**, **instantiates** ou **calls** entram no cálculo da ordem; referências puras de tipo (tipos de campos, tipos de parâmetros e retorno, casts) permanecem no modelo bruto, mas não são consideradas.

2. **Reconhecer SCCs:** No grafo de classes, componentes fortemente conexas são determinadas por Tarjan. O resultado ainda não é um layout, mas a informação de quais classes podem ser ordenadas de forma acíclica e quais pertencem a ciclos.
3. **Derivar o grafo de pacotes:** A partir das dependências entre classes, as dependências entre pacotes são agregadas em um grafo global ponderado de pacotes. Cada chamada de método de uma classe no pacote P para uma classe no pacote Q contribui para o peso da aresta $P \rightarrow Q$, propagada pela cadeia de ancestrais de P . O resultado é um único grafo plano abrangendo todos os pacotes.
4. **Calcular a ordem de pacotes:** A partir das entradas e saídas ponderadas calcula-se a medida de rank $rank(P)$. Todas as arestas para as quais $rank(P) < rank(Q)$ vale são classificadas como back edges e cortadas; SCCs residuais são tratadas novamente com ranks recalculados, até que o grafo de pacotes seja acíclico. Se todos os ranks dentro de uma SCC são iguais, todas as arestas internas são removidas; os níveis surgem então de dependências fora da antiga SCC.
5. **Determinar back edges de classes:** SCCs de classes são quebradas usando a hipótese de pacotes. *Caso A* – uma aresta $A \rightarrow B$ dentro de uma SCC é cortada quando $pkgLevel(A) < pkgLevel(B)$: ela corre contra a direção arquitetural. *Caso B* – quando todos os membros da SCC compartilham o mesmo nível de pacote, nenhum contexto de pacote fornece uma direção; todas as arestas internas são removidas. Em ambos os casos, *cortar* significa que a aresta fica de fora do DAG de cálculo dos níveis mas permanece preservada como achado de *classBackEdge*, visível como aresta tracejada de violação.
6. **Atribuir posições locais de exibição:** Após a atribuição dos níveis de classes, o *localLevel* de cada elemento é calculado. Dentro de cada contêiner pai, um grafo ponderado somente entre irmãos dos filhos diretos é construído; as back edges de classes do Passo 5 são excluídas; o grafo de irmãos é acíclico por construção; o caminho mais longo produz um *localLevel* por filho. O *localLevel* governa o posicionamento visual dentro da caixa pai e é também a base da detecção visível de violações (o *visual rank* sobre a cadeia de níveis aninhada descrita acima). O *architectureLevel* é o nível semântico global; ele aciona o layout e o invariante global de consistência – uma condição necessária, mas não suficiente, pois uma comparação global plana não detecta arestas aninhadas que sobem entre contêineres irmãos.
7. **Verificar consistência:** No fim, a representação pronta é verificada redundantemente contra as regras de consistência. Dependências normais, SCCs, back edges e violações são distinguidos e classificados.

Em resumo:

```
Bytecode -> grafo de classes -> SCCs de classes (overlay vermelho)
          -> grafo de pacotes -> SCCs / tangles de pacotes (overlay
              laranja)
          -> architectureLevel de pacotes
          -> architectureLevel de classes -> localLevel -> regras de
              consistencia
```

11 Conclusão

S202 calcula, a partir das dependências existentes, a hipótese mais plausível da arquitetura real. O resultado é uma representação hierárquica em camadas e verificável: dependências entre classes fornecem o grafo, dependências entre pacotes fornecem uma ordem robusta, back edges tornam visíveis as quebras necessárias, e regras de consistência verificam se a representação final cumpre sua própria afirmação.

Uma decisão de projeto fundamental é a separação explícita de dois tipos de ciclos. Ciclos de classes e ciclos de pacotes são detectados independentemente, armazenados como achados distintos e expostos por meio de chaves separadas na interface. Ciclos de classes (vermelho) mostram quais classes concretas formam cadeias cíclicas de dependência; ciclos de pacotes (laranja) mostram quais pacotes são mutuamente dependentes no nível arquitetural. Como ambas as visões estão disponíveis simultaneamente, a ferramenta pode responder a duas perguntas diferentes ao mesmo tempo: *onde* na estrutura de pacotes está o acoplamento circular, e *quais dependências concretas entre classes* o causam.

A contribuição mais importante, portanto, não está em uma figura visualmente melhorada, mas na conexão entre representação, explicação e capacidade de mudança. A estrutura em camadas mostra uma ordem possível. Os achados marcados explicam onde essa ordem é quebrada por dependências concretas. Movimentos what-if e CUTs manuais tornam visível quais refatorações podem reduzir essas quebras ou realmente dissolver ciclos.

Assim, a visualização é mais do que um layout. Ela é um modelo de trabalho explicável: cada posição e cada aresta destacada precisa poder ser rastreada até uma decisão compreensível no cálculo, e cada mudança planejada pode ser verificada contra a mesma ordem.